



Final Demo Environment Preparation

Building a stable, predictable showcase that highlights infrastructure, reliability, cost, and governance for your capstone project.

Why a Dedicated Demo Environment?



Stakeholders need a **safe, isolated sandbox** where you can:

- Protect production systems from risky experiments
- Ensure repeatable, reliable demo outcomes
- "Stage" scenarios like latency spikes, failover events, and model drift

A properly configured demo environment eliminates surprises during your presentation.

Characteristics of a Good Demo Environment

Isolated Namespace or Cluster

Complete separation from other environments ensures demo stability and predictability.

Pinned Versions

Lock down your model version, infrastructure components, and all configurations to prevent unexpected changes.

Synthetic but Realistic Test Data

Create data that showcases your system's capabilities without exposing sensitive information.

Stable Traffic Generator

Ensures dashboards display meaningful metrics and prevents "flatline" graphs during your presentation.

Pre-wired Dashboards & Reports

Configure all visualizations in advance for seamless navigation during the demo.

Infrastructure Setup: Namespace Isolation

```
kubectl create ns capstone-demo  
kubectl config set-context --current --namespace=capstone-demo
```

Create a clean, dedicated space for your demo that won't interfere with other environments.



Essential Configuration

Dedicated Service Accounts & RBAC

Ensure proper security boundaries and permissions

Resource Quotas

Prevent resource runaway situations during demo

Proper Labeling

env=demo, capstone_id=final

Model & API Deployment



Deploy Stable Container Image

Use a thoroughly tested build, not a last-minute creation

```
kubectl apply -f deploy/model-api.yaml
```



Pin Model Version

Explicitly use v1.3 instead of "latest" to ensure consistency

```
modelVersion: "1.3.0"
```



Enable Health Endpoints

Configure /healthz and /readyz for Kubernetes probes



API Documentation

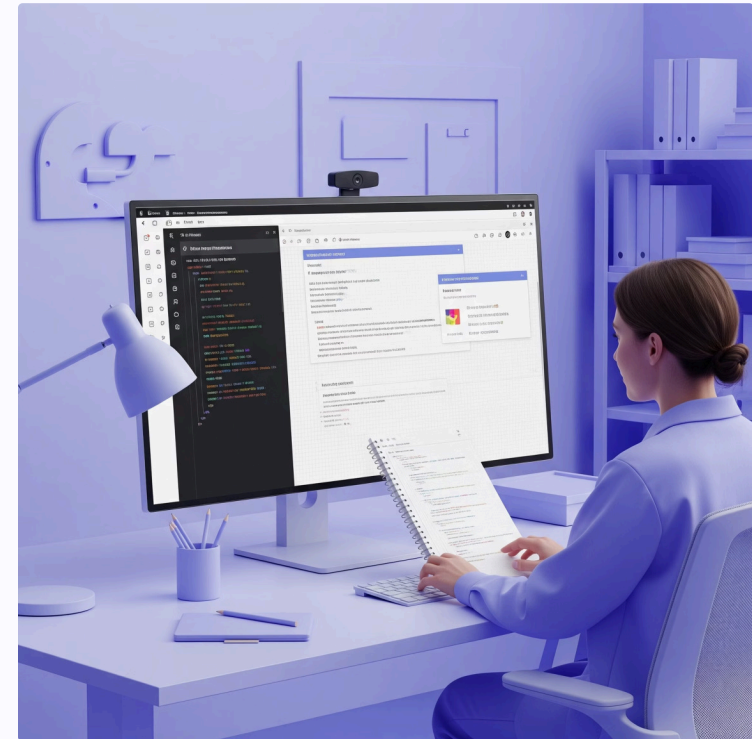
Make Swagger/ReDoc available at /docs for stakeholders

Pre-Load Data & Requests

Prepare Your Environment

- Seed [synthetic dataset](#) in feature store
- Create example payload templates for /predict endpoint
- Pre-run sample requests to warm up model cache
- Record "golden outputs" for comparison during demo

i **Pro tip:** Create a small script to validate that your pre-loaded data produces the expected outputs before the demo begins.



Traffic Generator for Demo Stability

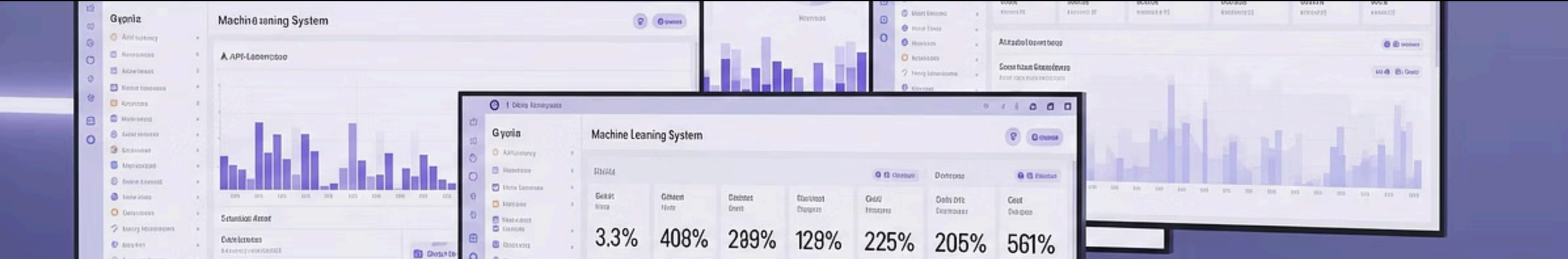
Run steady load to ensure dashboards show activity during your presentation:

```
k6 run --vus 20 --duration 5m script.js
```

Benefits

- Keeps [latency/error/cost panels](#) alive
- Prevents "flatline" dashboards during demo
- Demonstrates system behavior under realistic conditions
- Shows scaling capabilities in real-time





Observability Preparation

1

Essential Dashboards

Prepare browser tabs with:

- API Latency/Error metrics
- GPU Utilization panels
- Drift Report visualizations
- Cost Trend (Kubecost) overview

2

Annotations

Add timeline markers for:

- Demo start time
- Planned failover events
- Traffic pattern changes
- Key demonstration moments

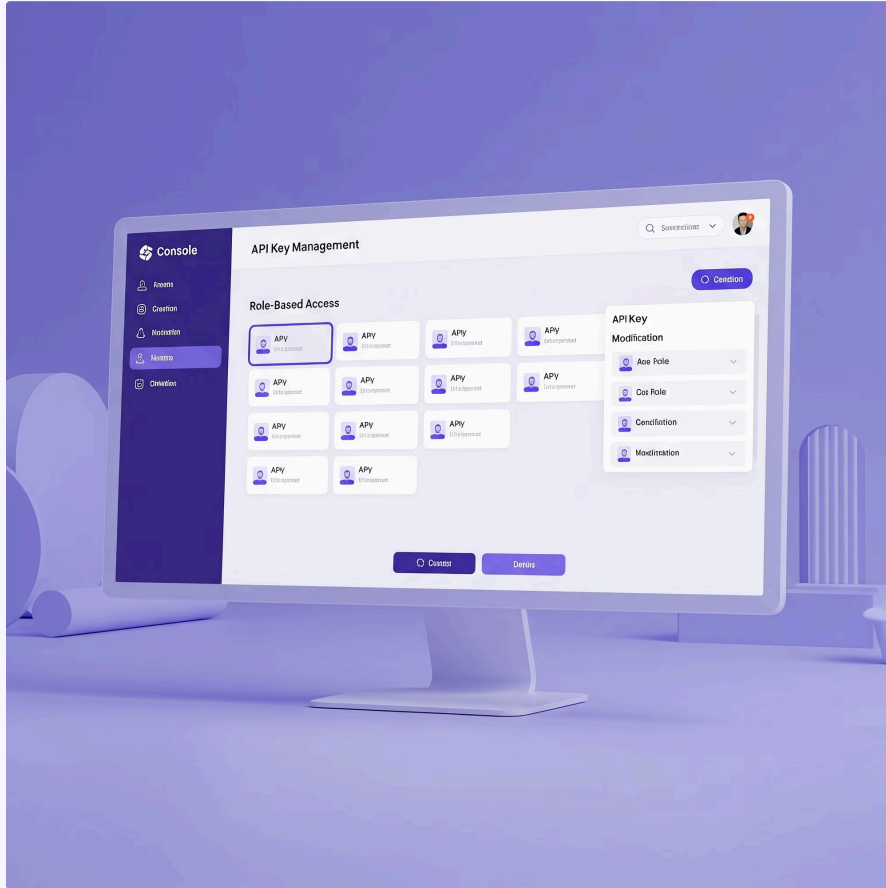
3

Presentation Tips

For maximum impact:

- Arrange dashboards in logical flow
- Set consistent time ranges (last 15 min)
- Use [theme colors](#) in custom panels
- Ensure legible font sizes for audience

Security & IAM for Demo



Secure Your Demo Environment

Demo API Key

Create short-lived credentials with appropriate quotas

Disable Dangerous Endpoints

Lock down retraining and other high-risk operations

Role-Based Access

Restrict environment to demo team members only

Audit Logging

Enable comprehensive activity tracking

Resilience Scenarios (Optional)

Add controlled drama to showcase your system's reliability:

High Availability Demo

Kill a pod and demonstrate seamless failover:

```
kubectl delete pod ml-api-7b9c8d6f5-2xv1z
```

Drift Detection

Inject synthetic drift to trigger alerts:

```
curl -X POST demo.api/inject-drift
```

Load Response

Spike traffic to show autoscaler in action:

```
k6 run --vus 100 --duration 30s spike.js
```

⚠ Important!

These scenarios should be [scripted for controlled demonstration](#), not true chaos. Practice each one multiple times before the actual demo.



Cost Guardrails in Demo



Demonstrating Cost Efficiency

Show stakeholders that your solution is both powerful and economical:

- Enable **budget alert** to warn if cost exceeds \$1 during demo
- Display **\$/1k inference** metric in Grafana dashboard
- Compare resource utilization against cost to prove efficiency
- Highlight cost optimization strategies implemented in your design

This demonstrates both technical excellence and business awareness.

Demo Script: Dry Run Checklist

API Validation

Call /predict endpoint and verify valid output in <300ms

```
curl -X POST https://demo.api/predict -d @sample.json
```

1

Metadata Check

/metadata shows correct model version and uptime

```
curl https://demo.api/metadata
```

2

Dashboard Review

Confirm Grafana shows active traffic + stable p95 latency

3

4

Drift Monitoring

Verify drift panel displays expected report format

Resilience Test

Kill pod and confirm no outage is observed

5

6

Rollback Verification

Test that rollback toggle works correctly (if included)

Backup Plan (Always Have One)

Even with perfect preparation, things can go wrong

Keep these fallbacks ready:



Local Notebook

API call screenshots and outputs



Grafana Snapshots

Saved dashboard states showing key metrics



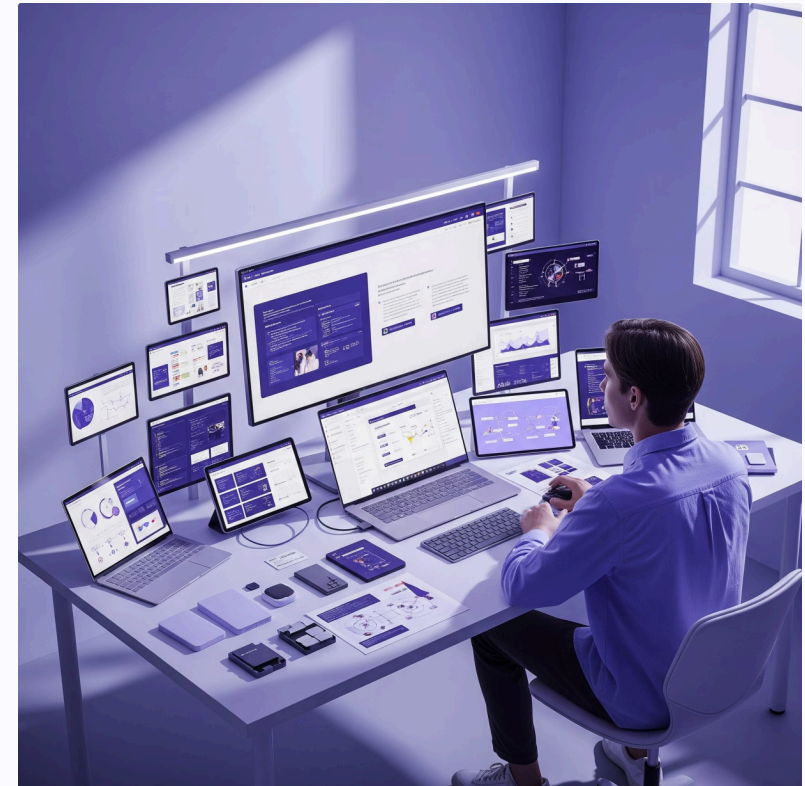
Pre-recorded Segments

Drift reports and failover demonstrations



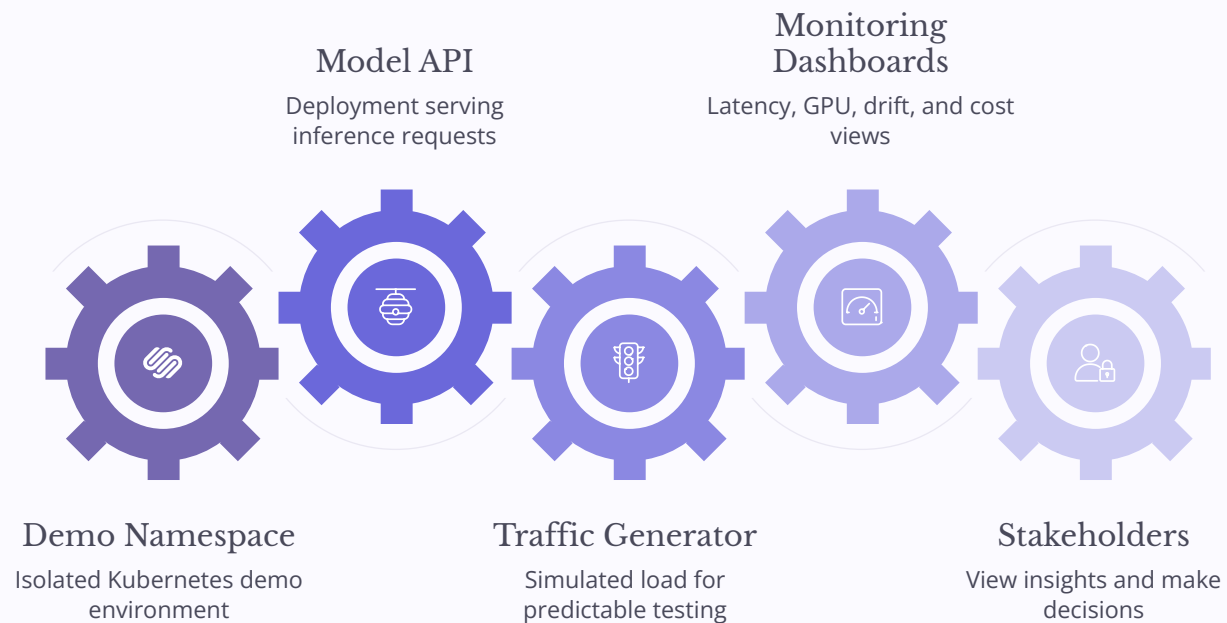
Offline Slides

Architecture diagrams and static metrics



- Remember: A smooth recovery from a minor issue can actually **impress stakeholders** more than a flawless demo, as it shows real-world resilience!

Demo Environment Architecture



Isolated, stable, story-driven demo setup designed for predictable performance

Summary: Keys to Demo Success



Isolation

Dedicated demo environment for predictability and safety



Stability

Pinned versions, pre-loaded data, consistent traffic



Visibility

Dashboards open and annotated in advance



Contingency

Always prepare backup materials



Your capstone demo should tell a [compelling story](#) about your system's capabilities while demonstrating technical excellence and attention to detail.

Discussion Prompt

"Would you rather demo a **live failover event** (more impressive, risky) or keep the demo **clean & controlled**? Why?"

Live Failover Considerations

- Higher impact when successful
- Demonstrates real resilience
- Shows confidence in your system
- Creates memorable moment

Clean Demo Considerations

- More predictable outcome
- Focuses attention on planned messages
- Reduces presentation anxiety
- Avoids potential embarrassment

